

5주차 예습과제

[4장]

- 분류 실습 - 캐글 산탄데르 고객 만족 예측

캐글의 산탄데르 고객 만족 데이터 세트에 대해, XGBoost와 LightGBM을 활용해 고객 만족 여부 예측하기 (<https://www.kaggle.com/competitions/santander-customer-satisfaction/rules>)

데이터 전처리

클래스 레이블 Target이 1이면 불만, 0이면 만족 고객이다. 모델의 성능 평가는 ROC-AUC로 이루어진다. (대부분 만족일 것이므로 정확도 수치보다 적합하다-'진짜 양성 비율(TPR)에 대한 거짓 양성 비율(FPR)')

모듈 로딩 + 학습데이터 DF로 로딩

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings

warnings.filterwarnings('ignore')
cust_df = pd.read_csv("./train_santander.csv", encoding='latin-1')
print('dataset shape:', cust_df.shape)
cust_df.head(3)
```

[Output]

dataset shape: (76020, 371)

ID	var3	var15	imp_ent_var16_ult1	imp_op_var29_comer_ult1	imp_op_var29_comer_ult3	imp_op_var40_comer_ult1	imp_op_var40_comer_ult3	imp_op_var40_
0	1	2	23	0.0	0.0	0.0	0.0	0.0
1	3	2	34	0.0	0.0	0.0	0.0	0.0
2	4	2	23	0.0	0.0	0.0	0.0	0.0

```
print(cust_df['TARGET'].value_counts())
unsatisfied_cnt = cust_df[cust_df['TARGET'] == 1].TARGET.count()
total_cnt = cust_df.TARGET.count()
print('unsatisfied 비율은 {:.2f}'.format((unsatisfied_cnt / total_cnt)))
```

[Output]

```
0    73012
1     3008
Name: TARGET, dtype: int64
unsatisfied 비율은 0.04
```

Target 값 = 0이 73012개, 1이 3008개이다. 대부분이 만족(0)이며 불만족 응답을 한 고객은 4%에 불과하다

먼저 사이킷런의 train_test_split을 활용해 학습 데이터와 테스트 데이터 세트를 분리한 뒤, 비대칭한 데이터 세트이므로 Target 값(=클래스) 분포도가 비슷하게 추출되었는지 확인해야 한다.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels,
                                                    test_size=0.2, random_state=0)

train_cnt = y_train.count()
test_cnt = y_test.count()
print('학습 세트 Shape:{0}, 테스트 세트 Shape:{1}'.format(X_train.shape, X_test.shape))

print('학습 세트 레이블 값 분포 비율')
print(y_train.value_counts()/train_cnt)
print('\n 테스트 세트 레이블 값 분포 비율')
print(y_test.value_counts()/test_cnt)
```

[Output]

```
학습 세트 Shape:(60816, 369), 테스트 세트 Shape:(15204, 369)
학습 세트 레이블 값 분포 비율
0    0.960964
1    0.039036
Name: TARGET, dtype: float64
```

```
테스트 세트 레이블 값 분포 비율
0    0.9583
1    0.0417
Name: TARGET, dtype: float64
```

학습 데이터셋과 테스트 데이터셋 모두 원본 데이터와 유사하게 전체 데이터의 약 4%가 불만족 값(Target=1)로 추출된 것을 확인할 수 있다. XGBoost의 조기 중단 검증 데이터셋으로 활용하기 위해 X_train과 y_train을 다시 쪼개 학습/검증 데이터셋으로 분리한다. (test_size=0.3)

XGBoost의 모델 학습과 하이퍼 파라미터 튜닝

XGBoost 모델을 생성하고 예측 결과를 ROC AUC로 평가하며, 학습 수행은 사이킷런의 XGBClassifier를 기반으로 한다.

```
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score

# n_estimators는 500으로, random state는 예제 수행 시마다 동일 예측 결과를 위해 설정.
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156)

# 성능 평가 지표를 auc로, 조기 중단 파라미터는 100으로 설정하고 학습 수행.
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric="auc", eval_set=[(X_tr, y_tr),
(X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[: , 1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

→ ROC AUC: 0.8429

앞서 분리한 학습 / 검증 데이터셋을 이용해 eval_set에 넣고, 조기 중단은 100회로 설정한 뒤 학습을 진행했다.

HyperOpt의 베이지안 최적화 기법을 활용한 XGBoost의 하이퍼 파라미터 튜닝

+) HyperOpt: ML 모델의 하이퍼 파라미터 튜닝에 베이지안 최적화를 적용할 수 있게 제공되는 대표적인 파이썬 패키지로, 주요 로직은 입력 변수명, 입력값의 검색 공간 설정/목적 함수 설정/목적 함수의 최솟값을 가지는 최적 입력값 유추

1. 검색 공간 설정

```
from hyperopt import hp

# max_depth는 5에서 15까지 1 간격으로, min_child_weight는 1에서 6까지 1 간격으로
# colsample_bytree는 0.5에서 0.95 사이, learning_rate는 0.01에서 0.2 사이 정규 분포된 값으로 검색.
xgb_search_space = {'max_depth': hp.quniform('max_depth', 5, 15, 1),
                    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
                    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2) }
```

2. 목적 함수 생성

```

from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

# fmin()에서 호출 시 search_space 값으로 XGBClassifier 교차 검증 학습 후 -1* roc_auc 평균 값을 반환.
def objective_func(search_space):
    xgb_clf = XGBClassifier(n_estimators=100, max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           colsample_bytree=search_space['colsample_bytree'],
                           learning_rate=search_space['learning_rate'])

```

```

# 3개 k-fold 방식으로 평가된 roc_auc 지표를 담은 list
roc_auc_list = [ ]

# 3개 k-fold 방식 적용
kf = KFold(n_splits=3)
# X_train을 다시 학습과 검증용 데이터로 분리
for tr_index, val_index in kf.split(X_train):
    # kf.split(X_train)으로 추출된 학습과 검증 index 값으로 학습과 검증 데이터 세트 분리
    X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
    X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

    # early stopping은 30회로 설정하고 추출된 학습과 검증 데이터로 XGBClassifier 학습 수행.
    xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=30, eval_metric="auc",
               eval_set=[(X_tr, y_tr), (X_val, y_val)])

    # 1로 예측한 확률값 추출 후 roc auc 계산하고 평균 roc auc 계산을 위해 list에 결과값 담음.
    score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[: , 1])
    roc_auc_list.append(score)

# 3개 k-fold로 계산된 roc_auc 값의 평균값을 반환하되,
# HyperOpt는 목적함수의 최솟값을 위한 입력값을 찾으므로 -1을 곱한 뒤 반환.
return -1*np.mean(roc_auc_list)

```

3. fmin()함수 호출해 최적 입력값 찾기

```

from hyperopt import fmin, tpe, Trials

trials = Trials()

# fmin() 함수를 호출. max_evals 지정된 횟수만큼 반복 후 목적함수의 최솟값을 가지는 최적 입력값 추출.
best = fmin(fn=objective_func,
            space=xgb_search_space,
            algo=tpe.suggest,
            max_evals=50, # 최대 반복 횟수를 지정합니다.
            trials=trials, rstate=np.random.default_rng(seed=30))

print('best:', best)

```


[Output]

```
100%|#####| 50/50 [23:34<00:00, 28.29s/trial, best loss: -0.8377636283189627]
best: {'colsample_bytree': 0.5749934608268169, 'learning_rate': 0.15145639274819528, 'max_depth': 5.0, 'min_child_weight': 6.0}
```

colsample_bytree가 약 0.5749, learning_rate가 약 0.1514, max_depth는 5.0, min_child_weight는 6.0이 도출됐습니다.

이제 이 최적 하이퍼 파라미터를 기반으로 분류 모델을 재학습시키고 테스트 데이터셋에서 ROC AUC를 측정한다. (n_estimators=500으로 증가)

```
# n_estimators를 500 증가 후 최적으로 찾은 하이퍼 파라미터를 기반으로 학습과 예측 수행.
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=round(best['learning_rate'], 5),
                        max_depth=int(best['max_depth']),
                        min_child_weight=int(best['min_child_weight']),
                        colsample_bytree=round(best['colsample_bytree'], 5)
                        )

# evaluation metric을 auc로, early stopping은 100으로 설정하고 학습 수행.
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100,
            eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:,:1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

→ ROC AUC: 0.8457

XGBoost가 GBM보다는 빠르지만 수행 시간이 상당히 오래 소요된다. 이는 GBM을 기반으로 하고 있기 때문이다. 앙상블 기반 알고리즘(배깅, 부스팅 등)에서 하이퍼 파라미터 튜닝으로 성능 수치가 급격히 개선되는 경우는 많지 않은데, 이는 앙상블 계열 알고리즘이 과적합이나 잡음에 기본적으로 우수한 알고리즘이라서 그렇다.

LightGBM을 이용한 모델 학습과 하이퍼 파라미터 튜닝

모델만 LightGBM으로 바꾸어 동일 작업을 수행한다. 검색 공간 설정, 목적 함수 생성, fmin 호출, 성능 평가 부분 모두 핵심 로직은 전부 동일하게 진행된다. (모델 객체와 하이퍼 파라미터명에 유의하기)

```
from lightgbm import LGBMClassifier

lgbm_clf = LGBMClassifier(n_estimators=500)

eval_set=[(X_tr, y_tr), (X_val, y_val)]
lgbm_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric="auc", eval_set=eval_set)

lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[:,:1])
print('ROC AUC: {0:.4f}'.format(lgbm_roc_score))
```

→ ROC AUC: 0.8384

1. 검색 공간 설정

```
lgbm_search_space = {'num_leaves': hp.quniform('num_leaves', 32, 64, 1),
                     'max_depth': hp.quniform('max_depth', 100, 160, 1),
                     'min_child_samples': hp.quniform('min_child_samples', 60, 100, 1),
                     'subsample': hp.uniform('subsample', 0.7, 1),
                     'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
                     }
```

2. 목적 함수 생성

```
def objective_func(search_space):
    lgbm_clf = LGBMClassifier(n_estimators=100,
                              num_leaves=int(search_space['num_leaves']),
                              max_depth=int(search_space['max_depth']),
                              min_child_samples=int(search_space['min_child_samples']),
                              subsample=search_space['subsample'],
                              learning_rate=search_space['learning_rate'])

    # 3개 k-fold 방식으로 평가된 roc_auc 지표를 담은 list
    roc_auc_list = []

    # 3개 k-fold 방식 적용
    kf = KFold(n_splits=3)

    # X_train을 다시 학습과 검증용 데이터로 분리
    for tr_index, val_index in kf.split(X_train):
        # kf.split(X_train)으로 추출된 학습과 검증 index 값으로 학습과 검증 데이터 세트 분리
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        # early stopping은 30회로 설정하고 추출된 학습과 검증 데이터로 LGBClassifier 학습 수행.
        lgbm_clf.fit(X_tr, y_tr, early_stopping_rounds=30, eval_metric="auc",
                    eval_set=[(X_tr, y_tr), (X_val, y_val)])

        # 1로 예측한 확률값 추출 후 roc auc 계산하고 평균 roc auc 계산을 위해 list에 결과값 담음.
        score = roc_auc_score(y_val, lgbm_clf.predict_proba(X_val)[:, 1])
        roc_auc_list.append(score)

    # 3개 k-fold로 계산된 roc_auc 값의 평균값을 반환하되,
    # HyperOpt는 목적함수의 최솟값을 위한 입력값을 찾으므로 -1을 곱한 뒤 반환.
    return -1*np.mean(roc_auc_list)
```

3. fmin() 함수 호출해서 최적 하이퍼 파라미터 도출

→

【Output】

```
best: { 'learning_rate': 0.08582271133758617, 'max_depth': 121.0, 'min_child_samples': 69.0, 'num_leaves': 41.0, 'subsample': 0.814895083027029}
```

learning_rate가 약 0.08592, max_depth가 121.0, min_child_samples가 69.0, num_leaves가 41.0, subsample이 0.91489로 도출되었습니다. 이제 이들 하이퍼 파라미터를 이용하

4. 하이퍼 파라미터를 이용해 LightGBM 학습 후 ROC-AUC 평가

```
lgbm_clf = LGBMClassifier(n_estimators=500, num_leaves=int(best['num_leaves']),
                           max_depth=int(best['max_depth']),
                           min_child_samples=int(best['min_child_samples']),
                           subsample=round(best['subsample'], 5),
                           learning_rate=round(best['learning_rate'], 5)
                           )

# evaluation metric을 auc로, early stopping은 100으로 설정하고 학습 수행.
lgbm_clf.fit(X_tr, y_tr, early_stopping_rounds=100,
             eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])

lgbm_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:,:1])
print('ROC AUC: {0:.4f}'.format(lgbm_roc_score))
```

→ ROC AUC: 0.8384